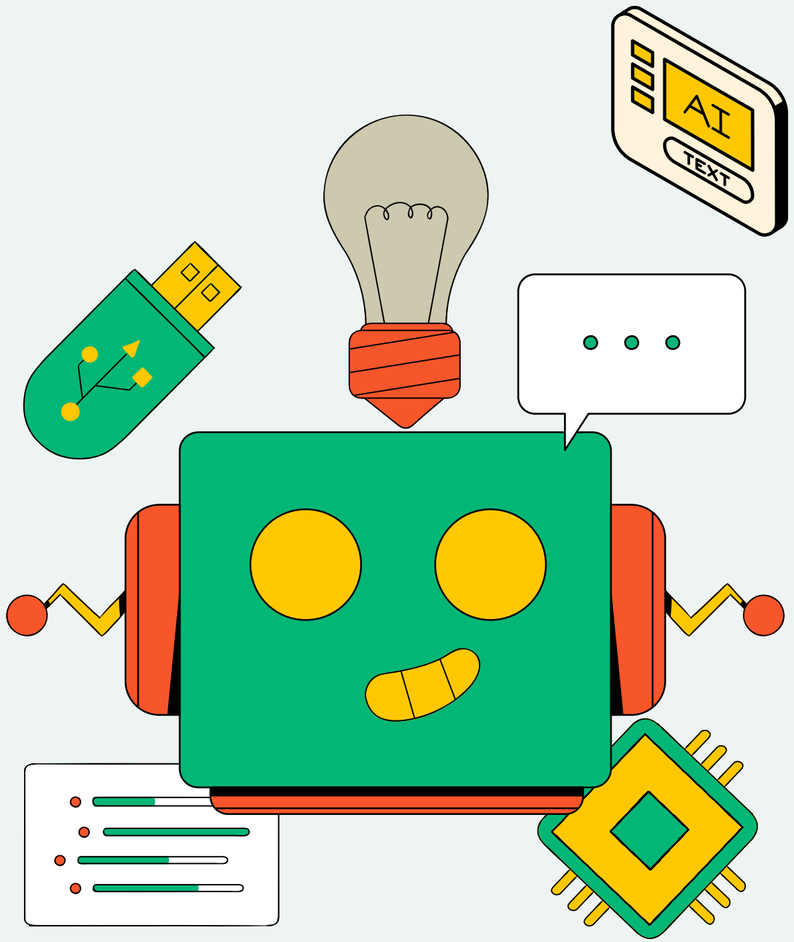
AYUDANTIA 7

MACHINE LEARNING



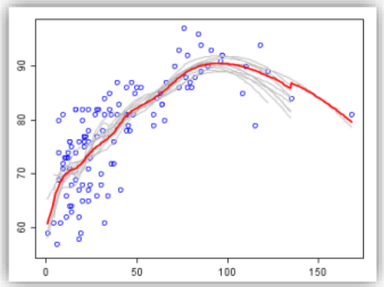
PRESENTED BY:

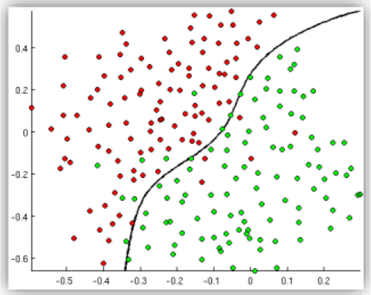
PATRICIO ACEVEDO

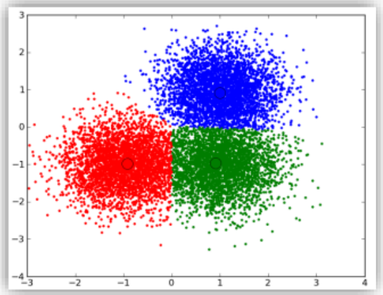
RECORDEMOS

KEY CONCEPTS

- CARACTERÍSTICAS = FEATURES = ATRIBUTOS = VARIABLES.
- ETIQUETA = LABEL = RÓTULO.
- CADA DATO (SIN SU ETIQUETA) PUEDE VERSE COMO UN VECTOR EN EL ESPACIO DE CARACTERÍSTICAS







DATOS Y SCIKIT-LEARN

Categóricos (cualitativos):

- **Nominales: Sin orden (ej. nombres, colores).**
- **Ordinales: Con orden (ej. nivel educativo, clasificaciones).**

Cuantitativos (métricos):

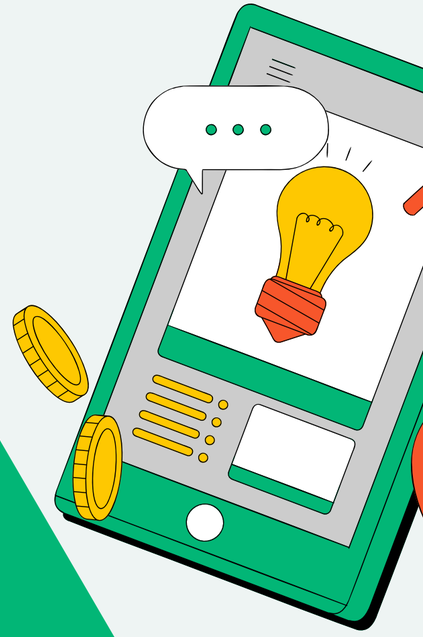
- **Discretos: Contables (ej. número de hijos).**
- **Continuos: Medibles (ej. peso, altura).**

Para completar nuestro laboratorio debemos usar el modelo que mas se adecue a nuestros datos, evaluando modelos candidatos usando nuestra libreria sklearn; el modulo a ejecutar es **sklearn.metrics**

¡¿ QUE MODELO ELEGIR ?!

- Datos mixtos
- Interpretabilidad
- Robustez y manejo de sobreajuste
- Escalabilidad
- Manejo de outliers

MODELOS MAS USADOS



Regresión Lineal

Árbol de Decisión

Random Forest

PCA (Análisis de Componentes Principales)

k-NN (k-Nearest Neighbors)

k-means Clustering (M. del codo)

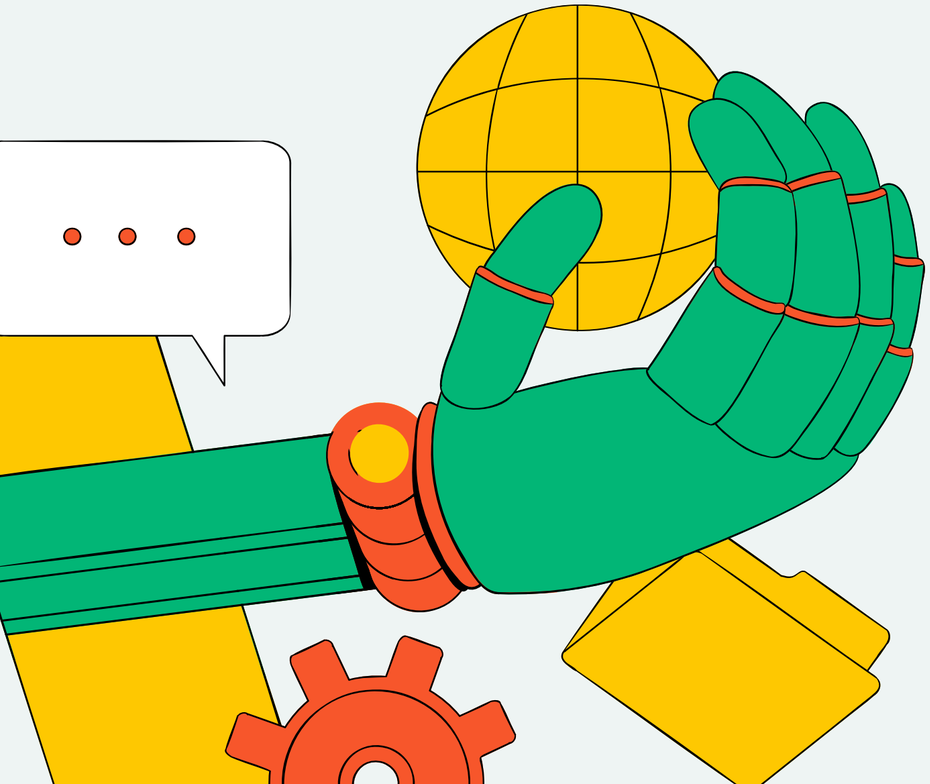
¡¿ YA PERO DEBEMOS HACER EN EL L3_1 ?!

- LIMPIAR NUESTROS DATOS

SIEMPRE QUE ESTOS SEAN DATOS NULOS (NO NECESARIAMENTE 0), QUE VIOLEN LAS RESTRICCIONES FUNDAMENTALES DE NUESTRO DB O QUE SENCILLAMENTE NO NOS SIRVAN EN NUESTRA INVESTIGACIÓN Y/O MODELO

• ELEGIR REPRESENTANTE

Este seguramente sea el punto mas importante del laboratorio, existen diversas formas dependiendo de cada modelo, algunos ejemplos

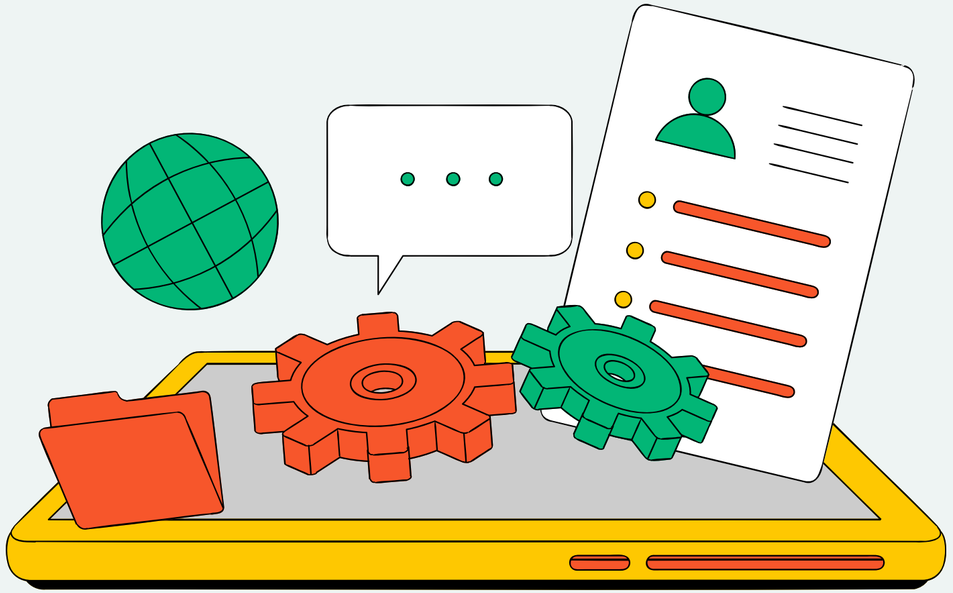


MEDIA

```
import numpy as np
representative_vector = np.mean(data, axis=0)
```

MEDIANA

```
import numpy as np
representative_vector = np.median(data, axis=0)
```



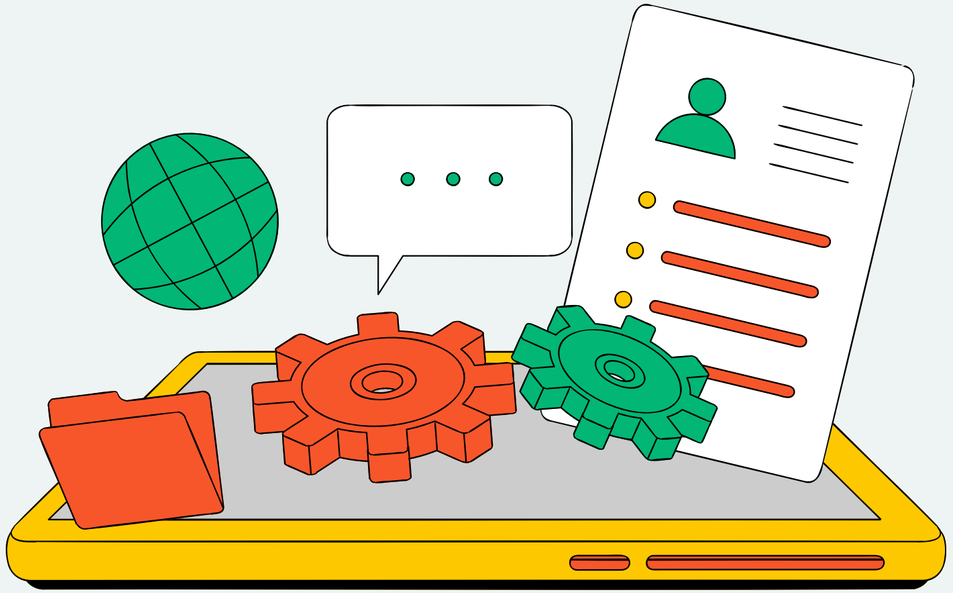
BASTANTE UTIL PARA :

IDEAL PARA DATOS CON OUTLIERS O DISTRIBUCIONES ASIMÉTRICAS.
ES ÚTIL PARA DATOS SIMÉTRICOS O SIN MUCHOS OUTLIERS.

ESTOS POR EJEMPLO PUEDEN SER USADOS PARA RELLENAR PAR DE DATOS QUE NO EXISTAN EN TU BASE DE DATOS EVITANDI DATOS EXTREMOS SIN PERDER MUCHA INFORMACION

CENTROIDES (K-MEANS)

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
kmeans.fit(data)
centroides = kmeans.cluster_centers_
```

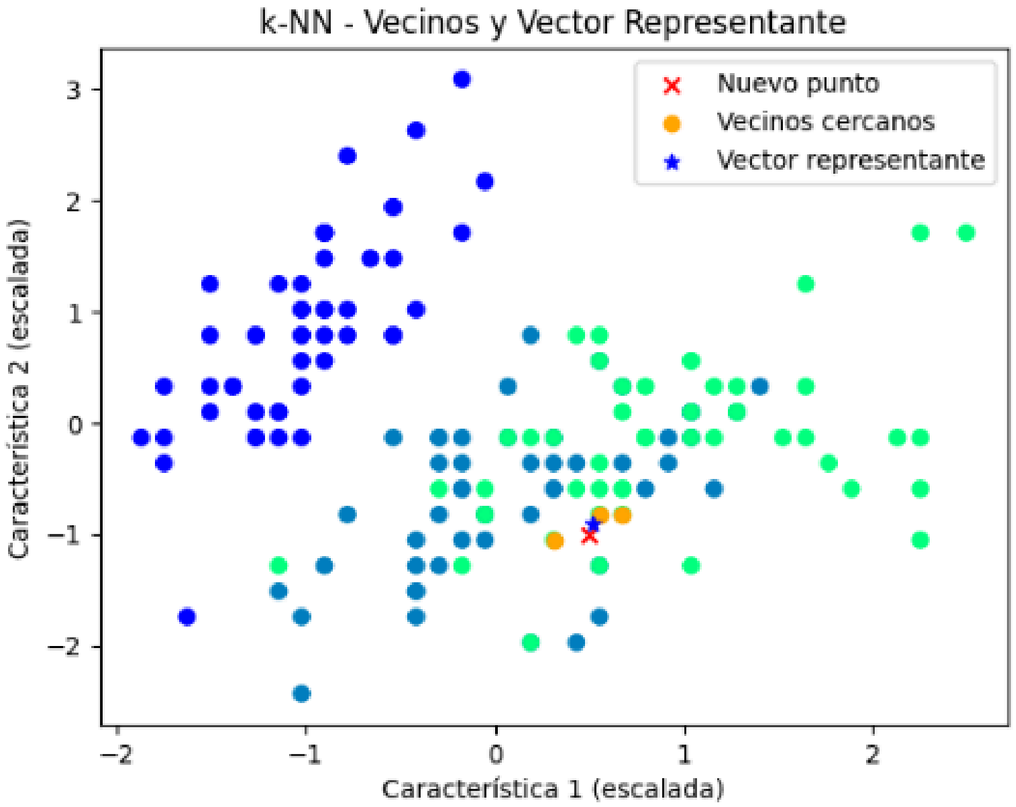


K-NN

```python
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Cargar el dataset Iris
iris = load_iris()
X = iris.data[:, :2]  # Usamos solo dos características para visualización
y = iris.target

# Estandarizar los datos
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Entrenar el modelo k-NN
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_scaled, y)

# Definir un nuevo punto para buscar sus vecinos más cercanos
new_point = np.array([[0.5, -1]])

# Encontrar los k vecinos más cercanos
distances, indices = knn_model.kneighbors(new_point)
nearest_neighbors = X_scaled[indices][0]

# Calcular el vector representativo (media de los vecinos más cercanos)
representative_vector = np.mean(nearest_neighbors, axis=0)
print("Vectores vecinos más cercanos:\n", nearest_neighbors)
print("Vector representante (media):\n", representative_vector)

# Visualización
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='winter')
plt.scatter(new_point[0, 0], new_point[0, 1], color='red', marker='x', label='Nuevo punto')
plt.scatter(nearest_neighbors[:, 0], nearest_neighbors[:, 1], color='orange', label='Vecinos cercanos')
plt.scatter(representative_vector[0], representative_vector[1], color='blue', label='Vector representante', marker='*')
plt.title('k-NN - Vecinos y Vector Representante')
plt.xlabel('Característica 1 (escalada)')
plt.ylabel('Característica 2 (escalada)')
plt.legend()
plt.show()
```

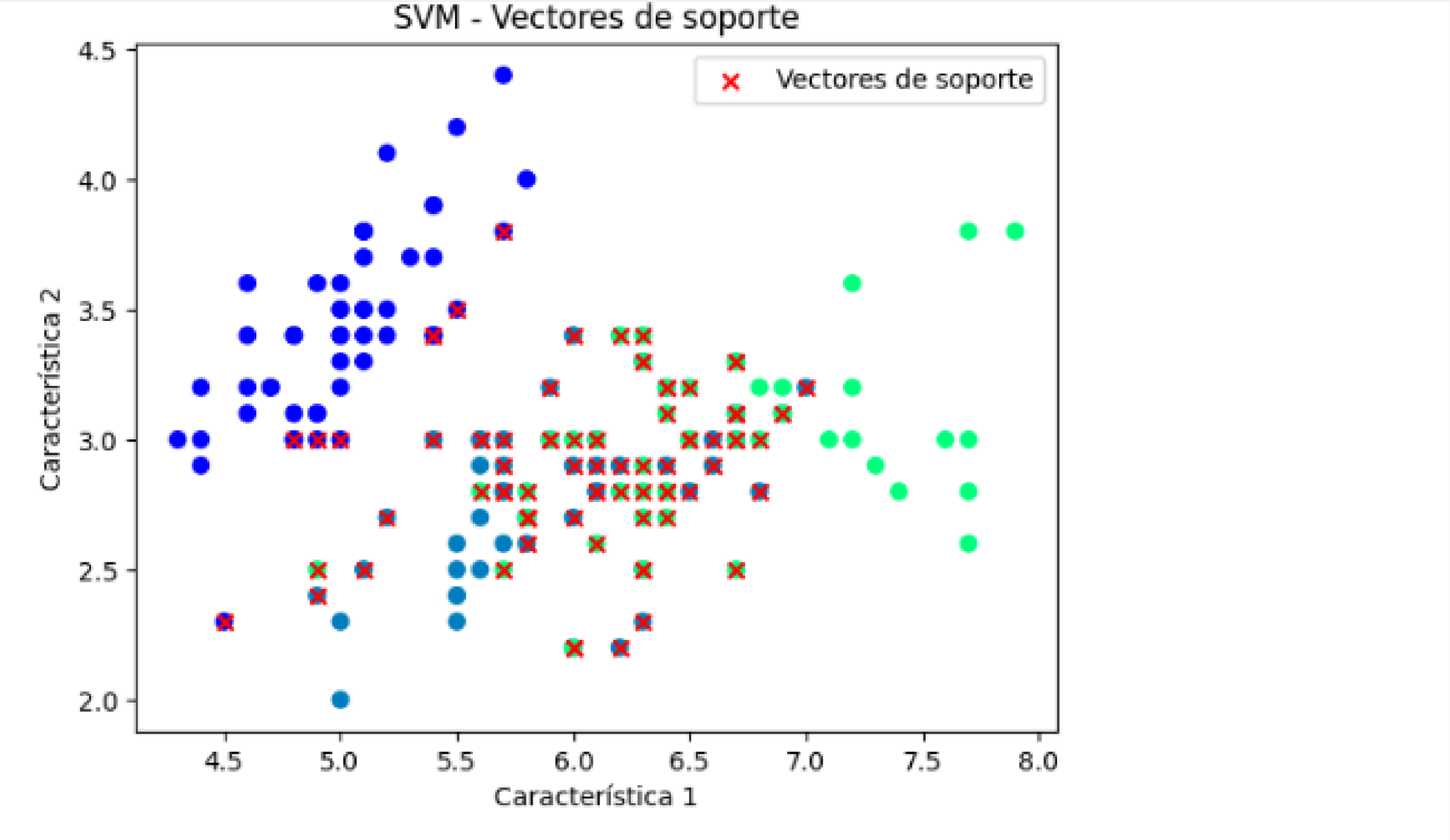
K-NN

```
Vectores vecinos más cercanos:
 [[ 0.55333328 -0.82256978]
 [ 0.31099753 -1.05276654]
 [ 0.67450115 -0.82256978]]
Vector representante (media):
 [ 0.51294398 -0.89930203]
```



SVM

```python
import numpy as np
from sklearn import datasets
from sklearn.svm import SVC
import matplotlib.pyplot as plt

# Cargar dataset (usaremos el dataset Iris para la clasificación)
iris = datasets.load_iris()
X = iris.data[:, :2]  # Usaremos solo dos características para visualización
y = iris.target

# Entrenar el modelo SVM
svm_model = SVC(kernel='linear')
svm_model.fit(X, y)

# Obtener los vectores de soporte
support_vectors = svm_model.support_vectors_
print("Vectores de soporte:\n", support_vectors)

# Visualizar los vectores de soporte
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='winter')
plt.scatter(support_vectors[:, 0], support_vectors[:, 1], color='red', marker='x', label='Vectores de soporte')
plt.title('SVM - Vectores de soporte')
plt.xlabel('Característica 1')
plt.ylabel('Característica 2')
plt.legend()
plt.show()
```


SVM - Vectores de soporte
Vectores de soporte
Característica 2
Característica 1
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

DIFICULTADES

SVM

- Dificultad en la selección de kernels:
- Sensible a outliers
- Difícil interpretación de los resultados
- Computacionalmente costoso para grandes datasets

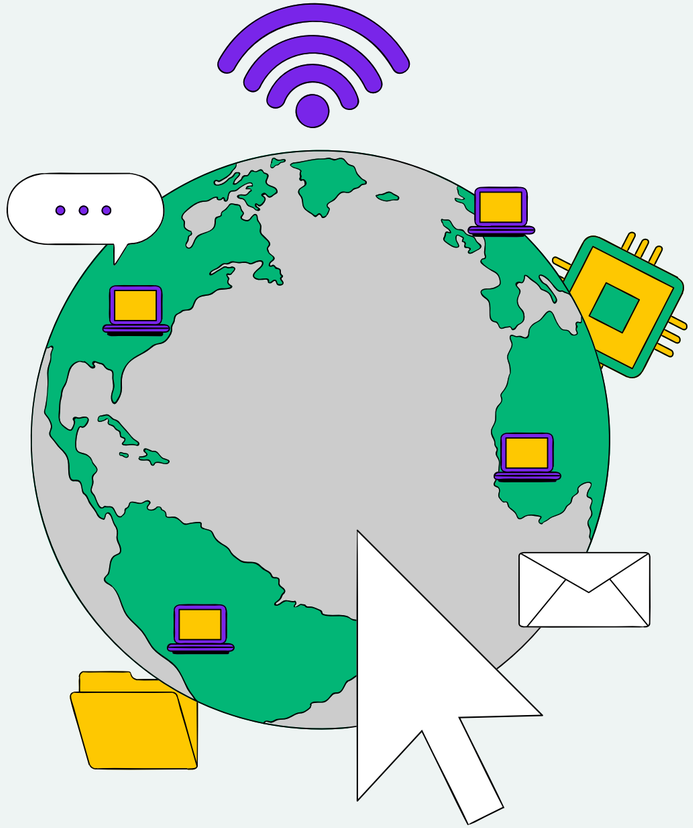
PUNTOS NEGATIVOS DE K-MEANS (CLUSTERING)

- Número de clústeres predefinido
- Sensibilidad a la inicialización
- Forma de los clústeres
- Sensible a outliers

K-NN (K-NEAREST NEIGHBORS)

- Computacionalmente costoso
- Sensibilidad a la elección de k
- Sensibilidad a la escala de los datos
- No captura bien la estructura de los datos en alta dimensionalidad

• ANALIZAR LOS RESULTADOS



El analizar los datos nunca será una tarea sencilla, desde entender los graficos hasta analizar si la solución es viable en la realidad..

Unos ejemplos:

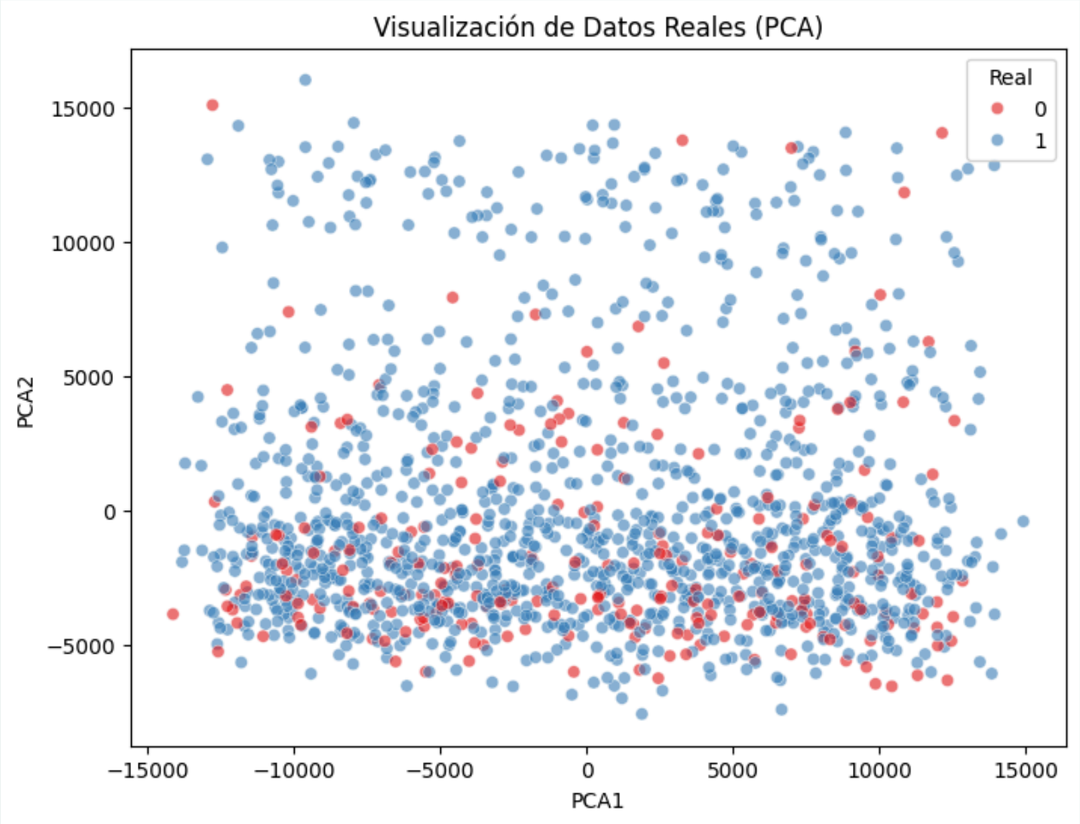
SVM for Spams emails

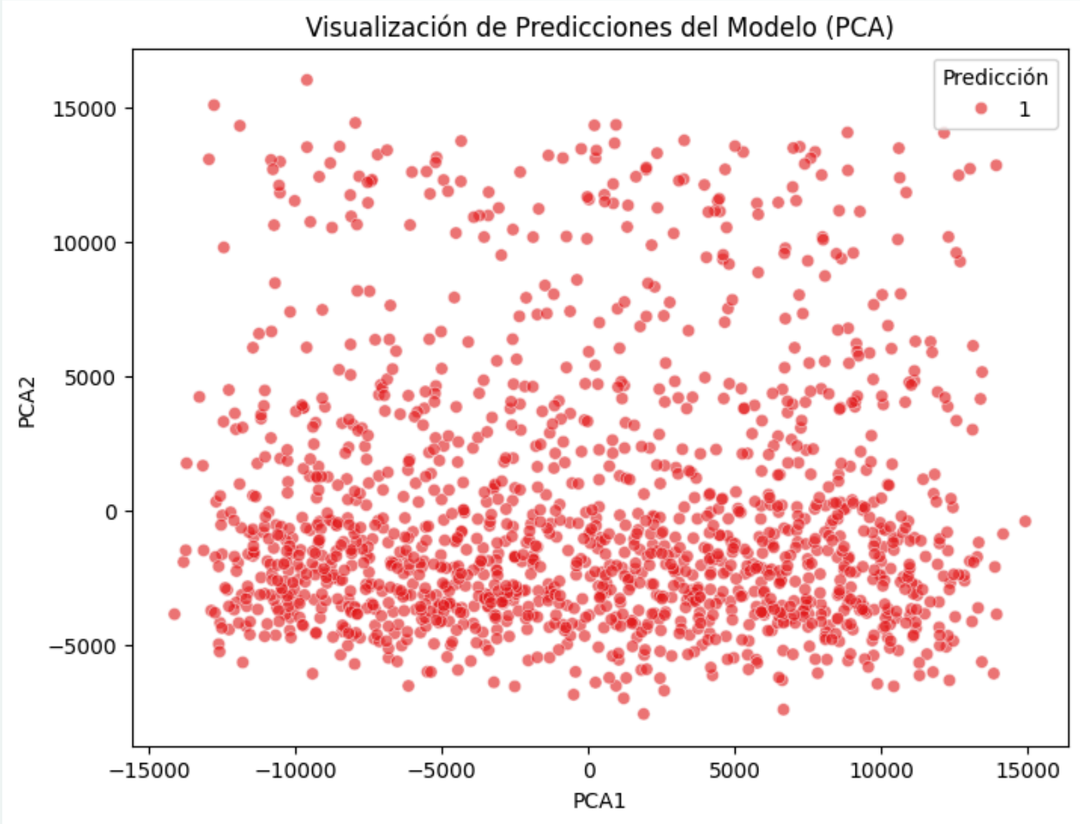
Google Flu Trends

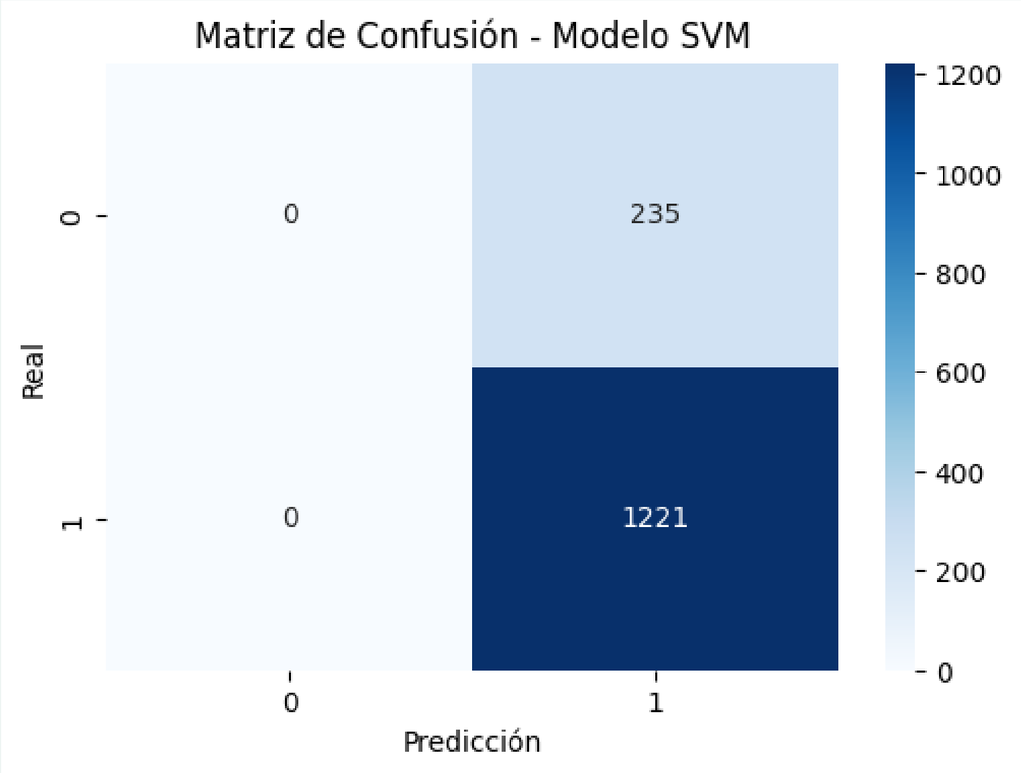
Sistema de detección de fallos en un entorno de computación en la nube (SVM)

Predicción de la morosidad de préstamos bancarios (k-NN y SVM)

Ejemplo:
Simulacion de despidos y contrataciones






Matriz de Confusión - Modelo SVM
Real
0
1
0
235
0
1221
0
1
Predicción
1200
1000
800
600
400
200
0

1. Matriz de Confusión

La matriz de confusión es una herramienta que permite visualizar el desempeño de un modelo de clasificación, mostrando los conteos de predicciones correctas e incorrectas en cada clase.

Gráfico:

Es un mapa de calor (heatmap) que muestra una matriz de 2x2, dado que tenemos dos clases:

- Clase 0: Empleados que no dejan la empresa (No Atrición).
- Clase 1: Empleados que dejan la empresa (Atrición).

Interpretación:

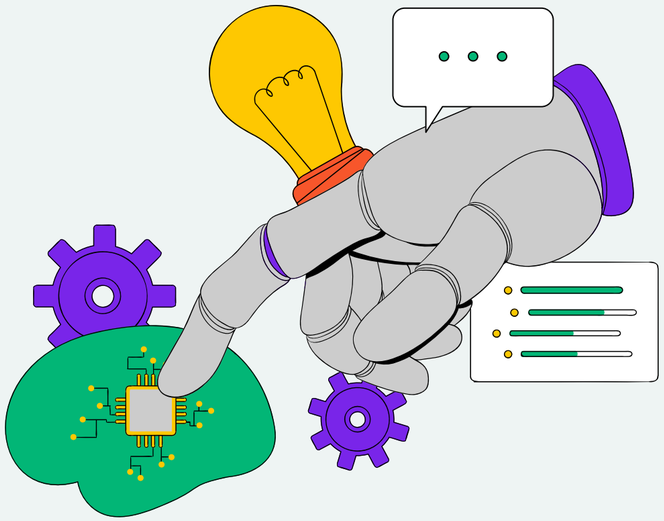
- Eje X (Predicción): Representa las clases predichas por el modelo.
- Eje Y (Real): Representa las clases reales en los datos de prueba.

Valores Observados:

- [0,0] - Verdaderos Negativos (VN): 0
  - Casos donde la clase real es 0 y el modelo predijo 0.
  - Indica que el modelo no predijo correctamente ningún empleado que se queda en la empresa.
- [0,1] - Falsos Positivos (FP): 874
  - Casos donde la clase real es 0 pero el modelo predijo 1.
  - Indica que el modelo clasificó erróneamente 874 empleados que se quedan como si fueran a dejar la empresa.
- [1,0] - Falsos Negativos (FN): 0
  - Casos donde la clase real es 1 pero el modelo predijo 0.
  - Indica que el modelo no clasificó erróneamente ningún empleado que deja la empresa.
- [1,1] - Verdaderos Positivos (VP): 411
  - Casos donde la clase real es 1 y el modelo predijo 1.
  - Indica que el modelo predijo correctamente los 411 empleados que dejan la empresa.

Conclusiones de la Matriz de Confusión:

- El modelo predice siempre la clase 1 (Atrición = Sí), sin importar la clase real.
- No es capaz de identificar ningún empleado que se queda en la empresa (clase 0).
- Esto refleja una incapacidad del modelo para distinguir entre las clases, lo que lo hace ineficaz para el propósito deseado.



Resumen de los Resultados:

- Los gráficos confirman que el modelo no está funcionando correctamente y que los resultados son irreales.

- El modelo predice que **todos los empleados dejarán la empresa**, lo que **es absurdo** y no viable desde un punto de vista empresarial.

COMPRENDAN EL MODELO y ANALICEN LOS RESULTADOS

GRACIAS POR VENIR !!

¿DUDAS?